

Risk & Technical Debt Inventory - Basanta Baral

Risk & Technical Debt Inventory

Individual Submission - Basanta Baral

Module: 1 – Senior Project II Reset: From Prototypes to Products

Student: Basanta Baral

Team: Howard University Senior Project Team

Date: February 3, 2026

Project: Synka - Biosimilar Switch Kit MVP

Repository: [GitHub](#) - [Synka MVP](#)

Executive Summary

This individual submission documents my contributions to the team's technical debt audit and risk assessment for the Synka MVP codebase. As we transition from Senior Project I to Senior Project II, I applied VIBE coding principles (Verify, Improve, Build, Execute) to analyze areas of the codebase I am responsible for and identify critical issues that must be addressed before scaling to production.

My analysis focuses on **backend security and infrastructure**, specifically: - Server-side input validation gaps (security vulnerability) - Database provider migration from SQLite to PostgreSQL - AI hallucination risks in medical eligibility logic

Part 1: Technical Debt Audit

My Assigned Technical Debt Items

Based on our team's code audit, I was assigned responsibility for the following technical debt items:

Debt Item 1: Missing Input Validation and Sanitization (Primary)

Attribute	Details
Item Name	Insufficient Server-Side Input Validation
Category	Architectural Debt (Security)
Severity	CRITICAL
Location	backend/src/controllers/, backend/src/middleware/validate.ts

Description:
While client-side validation exists via Formik/Yup in our React Native app, the backend lacks comprehensive server-side validation. Several endpoints accept user input without proper sanitization, creating potential security vulnerabilities including XSS attacks and data integrity issues. Given that we handle Protected Health Information (PHI), this is a critical gap.

Evidence from Codebase:

I reviewed the patient controller and found the following pattern:

```
// backend/src/controllers/patientController.ts
export const create = async (req: Request, res: Response) => {
  const { name, phone, dateOfBirth } = req.body;
  // No sanitization of name field - could contain scripts
  // No phone format validation on server
  // No date format validation
  const patient = await patientService.create(req.body);
  res.status(201).json(patient);
};
```

Why This Matters: - Healthcare applications require strict data validation to maintain integrity - Never trust client-side validation alone - it can be bypassed - PHI exposure through injection attacks would be a HIPAA violation - Malformed data could corrupt the database

Remediation Plan:

1. Install validation libraries:
- npm install express-validator xss-clean libphonenumber-js
2. Create validation schemas for each endpoint:

```
// backend/src/validators/patientValidator.ts
import { body, validationResult } from 'express-validator';

export const createPatientValidator = [
  body('name')
    .trim()
    .isLength({ min: 2, max: 100 })
    .escape(),
  body('phone')
```

```

    .isMobilePhone('any'),
    body('dateOfBirth')
    .isISO8601()
    .toDate(),
    body('diagnosis')
    .isIn(['RHEUMATOID_ARTHRITIS', 'CROHNS_DISEASE',
          'ULCERATIVE_COLITIS', 'PSORIASIS']),
  ];

```

3. Implement validation middleware:

```

// backend/src/middleware/validate.ts
export const validate = (req: Request, res: Response, next:
  NextFunction) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      error: 'Validation failed',
      details: errors.array()
    });
  }
  next();
};

```

4. Add rate limiting to prevent abuse:

```

import rateLimit from 'express-rate-limit';

export const apiLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  max: 100, // 100 requests per minute
  message: 'Too many requests, please try again later'
});

```

5. Testing plan:

- Test all endpoints with invalid input types
- Test with XSS payloads in string fields
- Test with oversized request bodies
- Verify proper 400 error responses

Estimated Effort: 5 story points

Priority: P0 - Critical

Sprint Target: Sprint 7

Debt Item 2: SQLite in Production Backend (Secondary)

Attribute	Details
Item Name	Development Database in Production Configuration
Category	Architectural Debt
Severity	HIGH

Attribute	Details
Location	backend/prisma/schema.prisma

Description:

The backend currently uses SQLite as its database provider. While SQLite was convenient for rapid prototyping during Senior Project I, it has significant limitations that make it unsuitable for production:

1. **No concurrent write support** - Multiple users could cause write conflicts
2. **Limited query capabilities** - Lacks advanced features for healthcare analytics
3. **Not suitable for multi-clinic deployment** - Each clinic would need separate instances
4. **No built-in encryption** - PHI would be stored unencrypted

Evidence from Codebase:

```
// backend/prisma/schema.prisma
datasource db {
  provider = "sqlite" // Development-only database
  url      = env("DATABASE_URL")
}
```

Why This Matters: - Production deployments require a robust database - PostgreSQL is the industry standard for healthcare applications - Concurrent access from multiple nurses/coordinators requires proper ACID compliance - Future analytics features require advanced query capabilities

Remediation Plan:**1. Set up PostgreSQL development environment:**

```
# docker-compose.yml
version: '3.8'
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: synka
      POSTGRES_PASSWORD: synka_dev
      POSTGRES_DB: synka_dev
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

2. Update Prisma schema:

```
datasource db {
  provider = "postgresql"
```

```
url      = env("DATABASE_URL")
}
```

3. Create environment-specific database URLs:

```
# .env.development
DATABASE_URL="postgresql://synka:synka_dev@localhost:5432/synka_dev"

# .env.production
DATABASE_URL="postgresql://user:password@production-
             host:5432/synka_prod"
```

4. Migration steps:

```
# Generate new migration
npx prisma migrate dev --name switch_to_postgres

# Seed development data
npx prisma db seed
```

5. Deploy to cloud provider:

- Option A: Railway PostgreSQL (\$5/month)
- Option B: Render PostgreSQL (free tier available)
- Option C: Supabase (free tier with 500MB)

Estimated Effort: 5 story points
Priority: P1 - High
Sprint Target: Sprint 7

Part 2: AI & System Risk Assessment

My Assigned Risk: AI Hallucination in Eligibility Logic

Attribute	Details
Risk Category	Reliability/Hallucination
Severity	CRITICAL
Likelihood	Medium
Impact	High - Could recommend unsafe medication switches

Description:
As the backend developer, I am responsible for the eligibility checker service that determines if a patient can safely switch from a reference biologic to a biosimilar. This logic was partially AI-generated during our rapid prototyping phase, and there is a significant risk that the AI hallucinated drug compatibility rules or missed critical contraindications.

In healthcare applications, incorrect eligibility decisions could lead to adverse patient outcomes. This is the most critical risk I identified in my area of responsibility.

Evidence of Risk:

I reviewed the eligibility logic in our codebase:

```
// backend/src/services/switchService.ts (AI-generated)
const DIAGNOSIS_DRUG_COMPATIBILITY = {
  'RHEUMATOID_ARTHRITIS': ['adalimumab', 'infliximab', 'etanercept'],
  'CROHNS_DISEASE': ['adalimumab', 'infliximab'],
  'ULCERATIVE_COLITIS': ['adalimumab', 'infliximab'],
  'PSORIASIS': ['adalimumab', 'etanercept'],
  // Question: Were these validated against FDA guidelines?
  // No source documentation exists
};

const ALLERGY_CONTRAINDICATIONS = {
  'ADALIMUMAB': ['latex', 'adalimumab'],
  'INFLIXIMAB': ['murine_proteins', 'infliximab'],
  // Question: Is this list complete?
  // Could AI have missed critical allergies?
};

const checkEligibility = (patient: Patient, targetDrug: string):
  EligibilityResult => {
  // Complex logic with multiple conditions
  // No documentation of business rules source
  // No version tracking for rule changes
};
```

Why This Risk is Critical: 1. **Patient Safety:** Incorrect eligibility could lead to adverse reactions 2. **Legal Liability:** Medical malpractice if harm occurs 3. **Trust:** Healthcare providers must trust our recommendations 4. **Regulatory:** FDA requires documented rationale for medical decisions

My Mitigation Strategy:

1. **Verification Protocol:** | Step | Action | Status | | — | — | — | — | — | — | | 1 | Cross-reference drug compatibility with FDA Purple Book | Pending | | 2 | Review contraindications with Cigna medical advisor | Pending | | 3 | Document source for each medical rule | Pending | | 4 | Add version control to eligibility rules | Pending |

2. Technical Safeguards I Will Implement:

```
// Add confidence scoring
interface EligibilityResult {
  eligible: boolean;
  confidenceScore: number; // 0-100
  requiresHumanReview: boolean;
  ruleVersion: string;
  sources: string[];
}

// Add audit logging
```

```

const logEligibilityDecision = async (decision: EligibilityResult)
  => {
  await prisma.eligibilityAuditLog.create({
    data: {
      patientId: decision.patientId,
      targetDrug: decision.targetDrug,
      eligible: decision.eligible,
      confidence: decision.confidenceScore,
      ruleVersion: decision.ruleVersion,
      timestamp: new Date(),
    }
  });
};

```

3. Testing Requirements:

- Unit tests for every eligibility rule
- Edge case testing for contraindications
- Negative testing (should reject ineligible patients)
- Medical advisor review of test cases

4. Monitoring Plan:

- Log all eligibility decisions for audit
- Alert on unusual patterns (e.g., >50% rejections)
- Weekly review of declined switches

Owner: Basanta Baral

Review Frequency: Weekly during Sprint 7

Part 3: Backlog Integration

GitHub Issues I Created

I converted my assigned technical debt items to GitHub Issues with AI-refined acceptance criteria:

Issue #46: [TECH-DEBT] Server-Side Input Validation

```

**Type:** Technical Debt
**Labels:** technical-debt, security, priority-critical
**Assignee:** Basanta Baral
**Sprint:** Sprint 7
**Story Points:** 5

```

Description

Server-side input validation is incomplete, creating security vulnerabilities for our healthcare application. This must be fixed before any pilot deployment.

Acceptance Criteria (AI-Refined)

- [] express-validator installed and configured
- [] Validation schema for POST /api/v1/patients
- [] Validation schema for POST /api/v1/switches
- [] Validation schema for POST /api/v1/follow-ups
- [] Validation schema for POST /api/v1/appointments
- [] Input sanitization middleware using xss-clean
- [] Phone number validation using libphonenumber-js
- [] Rate limiting middleware (100 req/min per IP)
- [] Request body size limits (1MB max)
- [] Validation errors return proper 400 responses with details
- [] All endpoints tested with invalid inputs (unit tests)
- [] Security review completed by team lead

Technical Approach

1. Create /backend/src/validators/ directory for validation schemas
2. Create reusable validation middleware in /backend/src/middleware/validate.ts
3. Reference existing Formik/Yup schemas from mobile app for consistency
4. Log validation failures for security monitoring

Definition of Done

- [] All acceptance criteria met
- [] PR approved by Cameron Carter (Tech Lead)
- [] Tests pass in CI/CD
- [] Documentation updated in API docs

Issue #47: [TECH-DEBT] Migrate to PostgreSQL

****Type:**** Technical Debt

****Labels:**** technical-debt, database, priority-high

****Assignee:**** Basanta Baral

****Sprint:**** Sprint 7

****Story Points:**** 5

Description

Backend uses SQLite which is unsuitable for production deployment.
Must migrate to PostgreSQL before pilot.

Acceptance Criteria (AI-Refined)

- [] Docker Compose file with PostgreSQL 15 configuration
- [] Prisma schema updated with provider = "postgresql"
- [] All migrations tested successfully on PostgreSQL
- [] Seed scripts work with PostgreSQL
- [] PostgreSQL deployed on Railway or Render for staging
- [] Environment variables configured (.env.development, .env.staging, .env.production)
- [] Data migration script created for any existing dev data
- [] Backup procedures documented

- [] Connection pooling configured (PgBouncer or built-in)
- [] Database indexes reviewed and optimized

Technical Approach

1. Create docker-compose.yml for local PostgreSQL
2. Update prisma/schema.prisma provider
3. Run prisma migrate dev to generate PostgreSQL migrations
4. Update seeding scripts if needed
5. Deploy to Railway free tier for staging
6. Document setup in README.md

Definition of Done

- [] All acceptance criteria met
- [] Backend tests pass with PostgreSQL
- [] Staging environment uses PostgreSQL
- [] Team can run locally with docker-compose up
- [] README updated with PostgreSQL setup instructions

VIBE Coding Verification

I applied the VIBE (Verify, Improve, Build, Execute) framework in my analysis:

Phase	My Application
Verify	Reviewed backend codebase to identify validation gaps and database limitations. Cross-referenced with HIPAA requirements and production best practices.
Improve	Created detailed remediation plans with specific code examples and steps. Identified root causes, not just symptoms.
Build	Converted debt items to GitHub Issues with AI-refined acceptance criteria. Created actionable, measurable tasks.
Execute	Prioritized items for Sprint 7. Committed to weekly progress reviews. Established clear definition of done.

Summary: My Contributions

Item	Type	Severity	My Role	Sprint
Server-Side Input Validation	Technical Debt	Critical	Primary Owner	Sprint 7
PostgreSQL Migration	Technical Debt	High	Primary Owner	Sprint 7
AI Hallucination	Risk	Critical	Risk Owner	Sprint 7

Item	Type	Severity	My Role	Sprint
in Eligibility Logic				

My Commitments for Sprint 7:

- 1. **Week 1:** Implement input validation for all backend endpoints
- 2. **Week 1:** Set up PostgreSQL with Docker Compose locally
- 3. **Week 2:** Complete database migration and update seed scripts
- 4. **Week 2:** Add unit tests for eligibility logic with medical review
- 5. **Ongoing:** Document all eligibility rules with FDA sources

Submitted By: Basanta Baral
Date: February 3, 2026
Course: Senior Project II
Module: 1 – From Prototypes to Products

This individual submission demonstrates my application of VIBE coding principles to analyze and address technical debt and AI-related risks in my areas of responsibility.